大模型宋词生成项目 PPT 完整内容稿

Slide 1 封面

标题

大模型宋词生成能力研究
格律、内容组织与内部同质化的综合分析

副标题

基于多模型、多词牌、多 prompt 条件的实验研究

页内信息

- 项目对象：AI 生成宋词与真实宋词基线
- 分析维度：格律、主题、意象、内部同质化
- 目标：回答“会不会写”“怎么写”“是否越写越像”

Slide 2 为什么这个项目值得做

标题

为什么要把“宋词生成”拆开研究

页内正文

当前关于古典诗词生成的讨论，往往停留在两个非常粗的判断上：

1. 像不像古诗词
2. 能不能写出通顺文本

但对于宋词而言，这两个判断远远不够。宋词写作至少同时牵涉四个层面：

- **格律是否成立**：词牌句式、字数、平仄、押韵是否基本可接受
- **题材是否得体**：不同词牌、不同题材是否出现相对恰当的表达组织
- **意象是否有内在系统**：是否真正形成对应题材的意象网络，而不是随意拼接
- **是否发生内部收缩**：模型在反复生成时，会不会越来越回到少数熟悉模板

因此，本项目不是单问“AI 会不会写宋词”，而是要进一步回答：

大模型在什么条件下能写得合格、写得像、写得稳定，又在什么地方最容易暴露模板化。

Slide 3 研究问题

标题

本项目要回答的四个核心问题

页内正文

问题一：格律能力

大模型能否在不同 prompt 条件下稳定生成格律基本合格的宋词?

问题二：默认写法

在不限定主题时，模型是否真的自由生成，还是会主动回到少数熟悉词牌和熟悉意象?

问题三：题材与意象的映射关系

不同 `category` 是否会稳定对应不同高频意象和偏好意象?

问题四：内部同质化

同一模型在相同条件下连续生成时，是否会在整词语义、句级表达与意象层面发生收缩?

Slide 4 实际使用的大模型

标题

参与实验的模型与样本规模

页内表格

模型原始名称	归一化名称	样本数	角色
doubao-seed-2-0-pro-260215	doubao-seed-2.0-pro	950	主模型
deepseek-reasoner + deepseek-v3.2	deepseek-3.2-thinking	949	主模型
qwen3.6-plus	qwen3.6-plus	801	主模型
glm-5	glm-5	211	辅助比较
qwen3.5-flash	qwen3.5-flash	50	`a1` 补充观察

模型原始名称	归一化名称	样本数	角色
qwen3-max	qwen3-max	48	`a1` 补充观察
glm-4.7	glm-4.7	45	`a1` 补充观察
kimi-k2.5	kimi-k2.5	28	`a1` 补充观察
gemini-3-flash	gemini-3-flash	20	少量样本
gemini-3.1-pro	gemini-3.1-pro	18	少量样本
gpt-5.4-high	gpt-5.4-high	2	极少量样本

页内说明

- 结论主要依赖三家主模型： `doubao / deepseek / qwen3.6-plus`
- 其余模型用于补充观察自由生成条件下的多样性和默认偏好

Slide 5 数据库与任务设计

标题

六个数据库不是六堆数据，而是一条逐步增强的约束链

页内表格

数据库	条件说明	Prompt 类型
`a1`	不限制词牌，不限制主题，自由生成	`type_c`
`b1`	固定五词牌，无主题	`type_b`
`b2`	固定五词牌，改革主题	`type_d`
`b3`	固定五词牌，玉兰主题	`type_d`
`c1`	固定 `沁园春` ，玉兰主题，显式格律模板	`type_a`
`c2`	固定 `沁园春` ，改革主题，硬性格律要求	`type_e`

页内说明

- `b1/b2/b3` 固定五个词牌： `菩萨蛮` / `沁园春` / `清平乐` / `祝英台近` / `浪淘沙`
- `c1/c2` 全部固定为 `沁园春`
- AI 总样本： `3122` 首

建议强调

这套设计允许我们依次观察：

- 放开词牌和主题时模型如何默认写
- 固定词牌后，结构压力如何改变结果
- 加入主题后，内容空间如何收缩
- 固定长调后，模型是否出现明显能力崩塌或模板化

Slide 6 `songci_engine` 的实现方法

标题

`songci_engine` 不是只做“对不对”判断，而是做“词牌召回 + 格律匹配 + 韵部解释”的综合校验

页内流程

`词谱.json / 韵书.json / 词林正韵.json -> 结构化载入 -> 词牌家族召回 -> 逐字校验 -> 韵脚通押判断 -> 最优变体输出`

页内正文

1. **数据层**：初始化时同时加载 `词谱.json` 、 `韵书.json` 、 `词林正韵.json` ，建立 `pattern_variants / rhyme_dict / rhyme_name_map / cilin_groups` 四组核心索引
2. **召回层**： `_get_candidate_patterns()` 会先提取词牌核心名，自动关联 `令 / 引 / 近 / 慢 / 减字 / 摊破` 等变体，避免模型词牌名稍有偏移就直接校验失败
3. **校验层**： `validate_prosody()` 先抽取纯汉字并检查字数，再按 `tunes` 逐字判断平仄、韵脚、句读和换句位置，同时单独处理未收录字与多音字
4. **押韵层**：优先按平水韵做严格押韵；如果窄韵未命中，再借助 `词林正韵` 做通押匹配，因此结果既保留传统约束，也允许较合理的宽韵解释
5. **输出层**： `evaluate()` 会在所有候选词牌及其格式中选择得分最高的一种，并返回 `score / errorCount / rhymeGroupLabel / matchedVariant / summary`

建议强调

这个引擎的价值不只是“判错”，而是把 AI 作品重新放回具体词牌系统中解释：它究竟错在字数、平仄、出韵，还是其实更接近另一个词牌变体。

Slide 7 Prompt 设计的核心差异

标题

不同 prompt 不只是措辞不同，而是在控制不同变量

页内表格

Prompt	核心特征	实验作用
`type_c`	模型自选词牌，无主题	观察默认词牌与默认内容路径
`type_b`	固定词牌，无主题	测试“词牌约束”本身的影响
`type_d`	固定词牌 + 固定主题 + 鼓励创造力	观察主题是否仍压缩表达空间
`type_a`	固定 沁园春 + 固定主题 + 显式格律模板	测试强结构提示的作用
`type_e`	固定 沁园春 + 固定主题 + 强调用韵格律	测试更短、更硬 prompt 的效果

页内结论

后面如果看到 `b2` 与 `c2`、`b3` 与 `c1` 有明显差异，这些差异不能只理解为题材效应，也必须理解为 prompt 结构效应。

Slide 8 完整 Prompt 文案如何处理

标题

主文展示 prompt 差异，完整原文放在附录

页内正文

因为观众对项目背景不熟悉，开头必须明确 prompt；
但因为观众对宋词专业，真正重要的不是机械读完 prompt，而是理解 prompt 怎样改变了“词体任务”的性质。

因此本 PPT 主文只强调：

- 哪些 prompt 固定了词牌
- 哪些 prompt 固定了主题
- 哪些 prompt 提供了格律模板
- 哪些 prompt 更强调创造力与文学性
- 哪些 prompt 是更硬性的格律任务

完整 `type_a / type_b / type_c / type_d / type_e` 原文见附录。

Slide 9 分析流程与评价框架

标题

从生成到评价：我们不是只算相似度

页内流程

`Prompt 设计 -> 批量生成 -> 数据入库 -> 格律检查 -> 主题/意象提取 -> 描述性统计 -> 同质化分析`

页内说明

本项目最终从四个层面评价模型：

1. **格律层**
2. **主题层**
3. **意象层**
4. **内部同质化层**

页内强调

这里的逻辑非常重要：
同质化并不是孤立模块，而是格律、题材与意象收缩最后在文本层面的表现。

Slide 10 格律结果：数据库级比较

标题

结果一：格律是第一道门槛， `c2` 是最严苛的压力测试

页内表格

数据库	平均格律分	中位数	100分占比	>=80占比	0分占比
a1	70.37	88	0.4901	0.5970	0.1990
b1	71.74	100	0.5452	0.6283	0.1953
b2	63.03	88	0.4013	0.5064	0.2404
b3	65.55	88	0.4433	0.5465	0.2521
c1	61.79	88	0.4156	0.5354	0.3140
c2	44.24	40	0.2273	0.3409	0.4508

页内结论

- `b1` 最稳定，说明固定词牌但不给主题，反而最有利于模型维持结构
- `c2` 最困难，说明“长调 + 固定主题 + 硬性 prompt”是最强压力场
- `c1` 明显高于 `c2` ，表明显式格律模板确实有帮助

Slide 11 格律结果：模型级比较

标题

结果二：真正拉开模型差距的，是长调与强约束下的格律控制能力

页内表格

模型	总样本	覆盖数据库数	平均格律分	>=80占比	0分占比
doubao-seed-2.0-pro	950	6	83.87	0.7547	0.0916
deepseek-3.2-thinking	949	6	56.02	0.4236	0.2940
qwen3.6-plus	801	6	53.17	0.4594	0.3061

补充比较： `c2`

模型	c2 平均格律分
doubao-seed-2.0-pro	79.76
qwen3.6-plus	28.68
deepseek-3.2-thinking	18.59

页内结论

如果只问“哪个维度最能拉开模型差距”，答案不是主题，而是格律，尤其是在 沁园春 这类长调和强约束条件下。

Slide 12 格律错误类型：问题主要出在哪

标题

格律失败的核心不是字数，而是平仄

页内表格

数据库	字数不符占比	应平实仄占比	应仄实平占比	出韵/挤韵占比
a1	0.1144	0.3507	0.2861	0.0522
b1	0.1472	0.2434	0.2682	0.0437
b2	0.1879	0.3694	0.3169	0.0414

数据库	字数不符占比	应平实仄占比	应仄实平占比	出韵/挤韵占比
`b3`	0.1946	0.3130	0.2724	0.0609
`c1`	0.1016	0.4483	0.3593	0.0436
`c2`	0.1402	0.6023	0.4924	0.0795

页内解读

- `c1/c2` 中平仄错误显著高于字数错误
- 模型不是完全不知道句式，而是难以持续维持长调中的细粒度声律规则
- 这对宋词研究尤其重要，因为它说明模型最难处理的不是“句子长度”，而是“词体内部的韵律连续性”

Slide 13 无主题条件下，模型并不自由

标题

结果三： `a1 / b1` 已经暴露出默认词牌和默认意象模板

页内表格一： `a1` 词牌选择

模型	作品数	词牌数	Top1 词牌	Top1 占比
qwen3-max	48	2	青玉案	0.8125
doubao-seed-2.0-pro	50	4	鹧鸪天	0.5200
qwen3.6-plus	49	9	鹧鸪天	0.4286
glm-5	50	18	鹧鸪天	0.2400

页内表格二： `a1 / b1` 数据库级高频意象

条件	高频意象（按 poem coverage）
`a1`	西风、孤舟、芦花、明月、星河、烟波、雁字、斜阳
`b1`	明月、孤舟、西风、暮云、雁字、斜阳、芦花、荻花

页内结论

- 自由生成不等于开放探索
- 模型会迅速回到少数熟悉词牌
- 内容上也会主动滑向少数熟悉的古典抒情模板

Slide 14 无主题条件下的模型差异

标题

不同模型有不同的“默认写法”

页内表格

条件	模型	`top10_imagery_ratio`	高频意象前 4 项
`a1`	qwen3-max	0.3088	芦花、西风、孤舟、烟水
`a1`	deepseek-3.2-thinking	0.1982	数据、代码、算法、星河
`a1`	qwen3.6-plus	0.1472	孤舟、雁字、西风、芦花
`a1`	doubao-seed-2.0-pro	0.0984	清露、岸柳、石径、晚风
`b1`	qwen3.6-plus	0.1363	孤舟、雁字、芦花、暮云
`b1`	deepseek-3.2-thinking	0.1098	明月、暮云、数据、星河
`b1`	doubao-seed-2.0-pro	0.0794	西风、疏钟、汀洲、明月

页内解读

- `qwen` 更容易回到旅愁模板
- `doubao` 更偏山水闲适与自然景物
- `deepseek` 更容易混入技术词或现代语汇残留

这不是简单的“语言风格不同”，而是 **默认意象资源池不同**。

Slide 15 不同 `category` 会稳定调用不同意象

标题

结果四：模型不是随机写题材，而是在调用题材模板

页内表格： `a1`

`a1` category	高频意象	偏好意象
离别羁旅	西风、芦花、孤舟、明月、烟波	黄芦、危樯、孤棹、孤篷、浮萍
闲适隐逸	明月、孤舟、松风、清露、雁字	幽篁、新茶、翠微、冰盏、寒泉
豪情壮志	星河、银汉、星斗、长空、数据	九霄、光年、天阙、悲歌、指尖

a1 category	高频意象	偏好意象
谈禅说理	星河、银河、算法、数据、云	芥子、须弥、云鹤、图灵、毫端

页内表格： `b1`

b1 category	高频意象	偏好意象
离别羁旅	孤舟、暮云、西风、明月、雁字	客衣、霜鬓、飘蓬、远影、霜空
闲适隐逸	明月、松风、斜阳、流水、青山	晨光、松窗、流泉、翠竹、荷气
怀古咏史	残碑、雁阵、芦花、西风、铁瓮城	铁马、废堞、渔樵、云楼、北固亭
悼亡相思	明月、阑干、孤灯、西风、霜风	孤枕、寒叶、尺素、笺痕、篱户

页内结论

AI 的无主题写作并不随机，而是会把不同题材压缩成几套可复用的意象模板。

Slide 16 真实主题设定一：改革主题到底要求模型写什么

标题

先看 prompt 中真正给出的主题，而不是只看数据库代号

页内主题原文

`b2` 与 `c2` 使用的是同一个主题设定：

> 假设你是一个修仙世界一个宗派的年轻长老，看到了宗派中传统的规矩和观念对宗派中的弟子的压抑，希望能改变这个现实，但是发现严格的规矩的确能一定程度上能保证在危机面前的战斗力，而且宗派中很多长老也很反对这个改革。你在挣扎中写了一首词来抒发自己的想法。

页内结果

条件	任务说明	最常见题材	占比	第二常见题材	占比
b2	固定五词牌 + 上述改革主题	谈禅说理	41.08%	咏物寄托	39.65%
c2	固定 沁园春 + 上述改革主题	谈禅说理	57.20%	豪情壮志	32.58%

页内高频意象

条件	最常见意象
b2	樊笼、劫火、青衿、灵根、春风、玄峰、仙踪、丹灶
c2	樊笼、青衿、仙峰、晨钟、灵根、丹峰、玄坛、旧规

页内说明

这个主题并不是现代政治改革，也不是现实制度批判，而是一个高度具体的“修仙宗门内部改革困境”。
模型写出来的作品，最后大多集中在几种稳定的表达方向上：

- 写旧规压人、理想难伸
- 写宗门中改革与秩序的两难
- 写弟子、长老、规矩、修炼之间的冲突

换句话说，模型并没有把这个主题写散，反而很快把它组织成一套稳定的修仙宗门叙事。

Slide 17 真实主题设定二：玉兰主题到底要求模型写什么

标题

玉兰主题并不只是“写花”，而是“借花写人生处境”

页内主题原文

`b3` 与 `c1` 使用的是同一个主题设定：

> 假设你是一个喜欢写宋词的大学生，最近在学业和面试上屡遇挫折，但是你看到了一颗盛开着白玉兰花的树，似乎重新找到了生命的实在与价值，便写了一首词。

页内结果

条件	任务说明	最常见题材	占比	第二常见题材	占比
b3	固定五词牌 + 上述玉兰主题	咏物寄托	92.05%	豪情壮志	5.25%
c1	固定 沁园春 + 上述玉兰主题	咏物寄托	87.66%	豪情壮志	10.89%

页内高频意象

条件	最常见意象
b3	玉兰、寒窗、素萼、暗香、书山、春寒
c1	玉兰、琼枝、寒窗、琼英、玉树、清芬

页内说明

这个主题的关键不是“白玉兰”本身，而是“受挫之后重新感到生命价值”。
但模型真正写出来时，几乎都转向了传统宋词里非常熟悉的一种写法：

- 先写玉兰的洁白、清芬、素萼
- 再把寒窗、书山、挫折、进取这些人生处境附着在玉兰之上
- 最终形成“借花自写”的咏物寄托结构

这也是为什么这一组条件下，“咏物寄托”的比例会高得异常稳定。

Slide 18 为什么这一部分要分析“内部同质化”

标题

这里真正想问的是：同一个模型写多了之后，会不会越来越像自己

页内正文

对第一次接触这个项目的人来说，“内部同质化”最容易被误解成“有没有抄袭”。
但本项目要测的不是这个。

这里要问的是一个更基础、也更适合生成模型研究的问题：

同一个模型，在同一种任务条件下反复写宋词时，作品之间会不会越来越接近，最后变成同一类词的不同改写版本？

这种“越来越像”大致有三种表现：

1. **整首词写得越来越像**
   虽然字句不同，但总是在写同一类情境、同一种情绪推进和同一类立意。
2. **句子越来越像**
   表面换了词，实际上句法骨架、修辞方式和表达次序反复出现。
3. **常用意象越来越固定**
   每次都反复回到同一组意象，不再真正调用新的资源。

所以，这里的“同质化”更接近“持续生成中的收缩”，而不是“单次生成中的失误”。

Slide 19 我们怎样判断“越来越像”

标题

同质化不是一个分数，而是三个层面的合看

页内正文

为了让这个问题更容易理解，本项目没有把所有结果压成一个黑箱分数，而是先分三层观察。

第一层：整首词是否越来越相近

这里看的不是某一句，而是整首词在主题组织、情绪推进和整体语义上的接近程度。

如果一组作品不断落回同一类叙事情境，比如总是“秋江离别”“寒窗受挫”“宗门旧规压人”，那么整首词层面的趋同就会升高。

第二层：句子是否越来越像

这里主要看句法骨架和表达路数是否反复出现。

例如：

- 同一组作品里经常出现相同的转折句法
- 总是先写景、再转情、再落理
- 或者同一种语义关系不断换词重写

第三层：意象是否越来越固定

这里看的是一组作品是否总是回到同一批意象。

如果写离别总是“孤舟、西风、雁字、芦花”，写玉兰总是“玉兰、寒窗、琼枝、清芬”，那么意象层的重复就会明显上升。

两个综合指标怎么理解

后面的表中会出现两个缩写：

- `core_mihi`：主要看“整首词相近”和“句子相近”
- `mihi_full`：在 `core_mihi` 的基础上，再把“意象重复”也计算进去

因此可以把它们简单理解为：

- `core_mihi`：作品在“写法”上有多像
- `mihi_full`：作品在“写法 + 常用意象”上有多像

分数越高，表示同一模型在同一条件下写出来的作品越容易互相靠近。

Slide 20 整体趋势：约束一层层增加，作品也一层层靠近

标题

从“固定五词牌、无主题”到“固定长调、固定主题”，同质化持续升高

页内表格

条件	主模型综合同质化指标范围（`mihi_full`）
固定五词牌、无主题（`b1`）	0.4394 - 0.5068
固定五词牌、改革主题（`b2`）	0.4644 - 0.5175
固定五词牌、玉兰主题（`b3`）	0.5181 - 0.5311
固定 沁园春 、玉兰主题（`c1`）	0.5957 - 0.6369
固定 沁园春 、改革主题（`c2`）	0.5925 - 0.6424

页内说明

这张表可以直接读出一条很清楚的顺序：

1. 只固定词牌、不固定主题时，同质化最低
2. 加入明确主题后，同质化开始上升
3. 再固定到单一长调 沁园春 ，同质化进一步显著上升

也就是说，限制越多，模型可走的路径越少，最后同一模型写出来的作品也就越容易互相靠近。

这与前面格律和题材结果是能够互相印证的：
一旦任务从“自由写一首词”变成“用固定长调、围绕固定主题写词”，模型就更容易把任务压缩成少数成熟套路。

Slide 21 先看最容易理解的一层：整首词为什么会越来越像

标题

主题加入后，模型首先收缩的往往不是句子，而是整首词的立意和叙事走向

页内正文

如果只看“从无主题到有主题”这一步，会发现一个很重要的现象：

- 先变得相似的，往往不是具体句子
- 而是整首词“在写什么”“怎么起承转合”

比如改革主题里，很多词最后都会回到：

- 旧规严苛
- 有志改革
- 改革有阻力
- 内心矛盾难解

玉兰主题里，很多词则回到：

- 先写玉兰之洁
- 再写现实困顿
- 最后把花与自我处境连起来

这说明当主题被固定后，模型首先被压缩的是“整首词的展开路数”。
所以我们后面会看到，很多模型在主题条件下的“整首词相近程度”提升很快，而“句子相近程度”未必立刻同步上升。

Slide 22 模型层比较：不同模型到底是怎样“越来越像”的

标题

三家主模型都存在收缩，但收缩得并不一样

页内表格

模型	综合同质化	整首词相近程度	句子相近程度	意象重复程度
doubao-seed-2.0-pro	0.563697	0.796665	0.522927	0.110865
qwen3.6-plus	0.541377	0.781168	0.476607	0.115194
deepseek-3.2-thinking	0.536261	0.757097	0.470848	0.153854

页内解读

doubao

`doubao` 的总体同质化最高。
它最突出的地方，是整首词和句子两个层面都比较容易靠近。

这意味着它在强任务条件下，往往会比较快地找到一种稳定写法，然后不断沿着这套写法生成。

`deepseek`

`deepseek` 的综合同质化不是最高，但“意象重复程度”最高。
这说明它最显著的问题不是句子一模一样，而是更容易反复回到同一组意象。

如果把它写玉兰或改革主题的作品放在一起看，会更容易感到：

- 不是字面上最像
- 但总是在调动同一批词、同一批景、同一批场景构件

`qwen3.6-plus`

`qwen3.6-plus` 的特点是：基线条件下已经比较相似。
换句话说，它不是在加了很多约束以后才开始收缩，而是在“固定五词牌、无主题”这种条件下就已经表现出较强的默认套路。

Slide 23 哪个模型最容易被主题带着走

标题

从“无主题五词牌”到“有主题五词牌”，不同模型的变化幅度并不一样

页内表格

模型	从无主题五词牌到改革主题五词牌，综合同质化增加值	从无主题五词牌到玉兰主题五词牌，综合同质化增加值
doubao-seed-2.0-pro	+0.026965	+0.040558
deepseek-3.2-thinking	+0.014119	+0.072638
qwen3.6-plus	+0.006895	+0.011278

页内说明

这张表不用看缩写，只看意思就够了：

- 把“固定五词牌、无主题”换成“固定五词牌、有主题”
- 看同一模型写出来的作品会不会更像彼此

从结果看：

- `deepseek` 最容易被主题拉向更稳定、更重复的写法，尤其是玉兰主题

- `doubao` 也会明显上升，但没有 `deepseek` 那么剧烈
- `qwen3.6-plus` 上升最小，说明它很多套路在无主题时已经存在

如果要把这页说得更直白一点，就是：

在三家主模型里， `deepseek` 最容易被明确主题“牵着走”。

Slide 24 同质化不是照抄，而是“换词不换骨架”

标题

为什么不能把同质化简单理解成重复句子

页内例子：玉兰主题

- 嫩蕊凝霜 vs 琼苞乍绽
- 素蕊含霜 vs 玉葩凝露
- 寒凝素骨 vs 玉质冰肌

页内例子：改革主题

- 锋藏剑鞘 vs 剑锷藏锋
- 严规立矩 vs 规矩虽严
- 严规峻矩 vs 规矩虽严

页内解读

这些句子并不是逐字重复，但它们有几个共同点：

1. 说的是同一种意思
2. 使用的是很相近的修辞关系
3. 落点都在同一类意象和同一类句法位置上

所以，本项目里看到的“越来越像”，很多时候不是“照抄”，而是：

同一套写法骨架，反复换词使用。

这也是为什么如果只看字面重复，很多问题会被漏掉；必须把整首词、句子和意象三层合在一起看。

Slide 25 综合结论

标题

这项研究最终说明了什么

页内结论

1. **格律是第一道门槛。**
   真正最能拉开模型差距的是格律，尤其是长调和强约束条件。
2. **自由生成并不自由。**
   在 `a1 / b1` 中，模型已经会回到少数熟悉词牌和熟悉意象模板。
3. **题材会系统性压缩写作路数。**
   不同 `category` 会稳定对应不同意象模板，固定主题会进一步强化这种映射。
4. **内部同质化真实存在，并随约束增强而上升。**
5. **不同模型的收缩机制不同。**
   `doubao` 更偏整词和句级收缩， `deepseek` 更偏常用意象反复出现， `qwen` 则更像基线阶段就已经较强模板化。

收束句

因此，评价大模型的宋词生成能力，不能只问“像不像宋词”，还必须问：

它是否真正掌握了词体的结构要求，是否具备题材组织能力，以及是否在持续生成中越来越重复自己。

Slide 26 局限与下一步

标题

局限与后续工作

页内正文

当前局限

- 主模型之外的部分模型样本量偏小
- `category` 与 `imagery` 提取仍存在自动标注误差
- 同质化已经量化完成，但案例化展示还可以继续加强

下一步工作

- 做更直观的 `category -> imagery` 可视化
- 做模型级模板簇或案例对照页
- 把现有结果进一步转为正式答辩版 PPT
- 必要时加入更多人工校验样本

附录

Appendix A 完整 Prompt 原文说明

完整 prompt 原文见 prompts.py。

为保证主文节奏，主文只保留 prompt 差异摘要。
如果现场需要展示全文，可在附录页中直接贴出以下五段：

- `type_a`
- `type_b`
- `type_c`
- `type_d`
- `type_e`

Appendix B 需要时可展开的完整 Prompt 原文

type_a

你现在是一位精通宋词的中国古典文学专家，也是一个数据生成引擎。
你需要使用词牌沁园春，并根据主题和背景，创作一首完全符合格律的完整的宋词。请务必注意所作词格律的准确性。内容希望
【格律参考】
中仄平平，仄仄平平，仄仄仄平。

仄中平中仄，中平中仄，中平中仄，中仄平平。

中仄平平，中平中仄，中仄平平中仄平。

平平仄，仄中平中仄，中仄平平。

平平中仄平平，仄中仄平平中仄平。

仄中平中仄，中平中仄，中平中仄，中仄平平。

中仄平平，中平中仄，中仄平平中仄平。

平平仄，仄中平中仄，中仄平平。

【输出要求】
你必须严格输出且仅输出合法的 JSON 格式数据，不要包含任何额外的解释、问候或 markdown 标记（如 ```json）。

【字段及约束说明】
1. "cipai" (字符串): 词牌名。
2. "title" (字符串): 词的自拟题目。
3. "content" (字符串): 词的正文。请严格遵循该词牌的字数和句式，句子之间用全角逗号“，”或句号“。”分隔，不要包含换

【示例】
输出：
```
{
  "cipai": "如梦令",
  "title": "春晚",
  "content": "昨夜雨疏风骤，浓睡不消残酒。试问卷帘人，却道海棠依旧。知否，知否？应是绿肥红瘦。",
}
```

输入：主题和背景：{theme}。现在，请开始生成。

type_b

你现在是一位精通宋词的中国古典文学专家，也是一个数据生成引擎。
你需要使用我提供的【词牌名】，创作一首符合格律的完整的宋词,使用词林正韵,使用钦定词谱。请务必注意所作词的字数，保

【输出要求】
你必须严格输出且仅输出合法的 JSON 格式数据，不要包含任何额外的解释、问候或 markdown 标记（如 ```json）。

【字段及约束说明】
1. "cipai" (字符串): 词牌名。
2. "title" (字符串): 词的自拟题目。
3. "content" (字符串): 词的正文。请严格遵循该词牌的字数和句式，句子之间用全角逗号“，”或句号“。”分隔，不要包含换

【示例】
输出：
```
{
  "cipai": "如梦令",
  "title": "春晚",
  "content": "昨夜雨疏风骤，浓睡不消残酒。试问卷帘人，却道海棠依旧。知否，知否？应是绿肥红瘦。",
}
```

现在，请开始生成。输入：词牌名：《{cipai_name}》

type_c

你现在是一位精通宋词的中国古典文学专家，也是一个数据生成引擎。
你需要自己选择一个词牌，创作一首符合格律的完整的宋词。请务必注意所作词的字数，格律（使用词林正韵、使用钦定词谱）

【输出要求】
你必须严格输出且仅输出合法的 JSON 格式数据，不要包含任何额外的解释、问候或 markdown 标记（如 ```json）。

【字段及约束说明】
1. "cipai" (字符串): 词牌名。
2. "title" (字符串): 词的自拟题目。
3. "content" (字符串): 词的正文。请严格遵循该词牌的字数和句式，句子之间用全角逗号“，”或句号“。”分隔，不要包含换

现在，请开始生成。

type_d

你现在是一位精通宋词的中国古典文学专家，也是一个数据生成引擎。
你需要使用我提供的【词牌名】和【主题和背景】，创作一首符合格律的完整的宋词，使用词林正韵，最好使用钦定词谱。请务

【输出要求】
你必须严格输出且仅输出合法的 JSON 格式数据，不要包含任何额外的解释、问候或 markdown 标记（如 ```json）。

【字段及约束说明】
1. "cipai" (字符串): 词牌名。
2. "title" (字符串): 词的自拟题目。
3. "content" (字符串): 词的正文。请严格遵循该词牌的字数和句式，句子之间用全角逗号“，”或句号“。”分隔，不要包含换

【示例】
输出：
```
{
  "cipai": "如梦令",
  "title": "春晚",
  "content": "昨夜雨疏风骤，浓睡不消残酒。试问卷帘人，却道海棠依旧。知否，知否？应是绿肥红瘦。",
}
```

现在，请开始生成。输入：词牌名：《{cipai_name}》，主题和背景：{theme}。

type_e

你现在是一位精通宋词的中国古典文学专家，也是一个数据生成引擎。
你需要使用词牌沁园春，并根据主题和背景，使用词林正韵,使用钦定词谱，创作一首完全符合格律的完整的宋词。

【输出要求】
你必须严格输出且仅输出合法的 JSON 格式数据，不要包含任何额外的解释、问候或 markdown 标记（如 ```json）。

【字段及约束说明】
1. "cipai" (字符串): 词牌名。
2. "title" (字符串): 词的自拟题目。
3. "content" (字符串): 词的正文。请严格遵循该词牌的字数和句式，句子之间用全角逗号“，”或句号“。”分隔，不要包含换

【示例】
输出：
```
{
  "cipai": "如梦令",
  "title": "春晚",
  "content": "昨夜雨疏风骤，浓睡不消残酒。试问卷帘人，却道海棠依旧。知否，知否？应是绿肥红瘦。",
}
```

输入：主题和背景：{theme}。现在，请开始生成。